

Task 2A - Search and Indexing Subsystem Design Smells

1. High Cognitive Capacity (The "Brain Method" Smell)

Evidence: SonarQube flagged cognitive complexity scores of 21–22 in `LuceneIndexManager`, while Designite reported 56 instances of Insufficient Modularization and 99 Complex Methods.

- **The Smell:** This occurs when a class or method takes on too many responsibilities that should have been decomposed into smaller, specialized abstractions. In the Lucene folder, the `LuceneIndexManager` is acting as a "Brain Class," handling everything from file-system locking to search parsing.
- **Design Impact:** This leads to High Fragility. Because the logic is not modularized, a change in how Lucene handles "write locks" requires modifying the same complex method that handles "search queries," increasing the risk of bugs.

2. Deficient Encapsulation (Missing Information Hiding)

Evidence: SonarQube repeatedly flagged missing private constructors and exposed visibility in `FieldConstants`, `IndexOperation`, and `ReadFromIndexOperation`. This is backed by Designite's 57 instances of Deficient Encapsulation.

- **The Smell:** The Lucene module is failing to hide its internal implementation details. By exposing constructors and internal operation classes publicly, the folder violates the Principle of Least Privilege.
- **Design Impact:** This creates inappropriate Intimacy between the search module and the rest of Apache Roller. It allows external packages to bypass the intended `LuceneIndexManager` API and manipulate index operations directly, making it nearly impossible to swap or upgrade the Lucene version without breaking the entire application.